

Homework 3

级数求和、机器精度与舍入误差实验报告

姜玥晟

2026-04-21



项目	内容
源题编号	HW03
学生姓名	姜玥晟
报告主题	Gregory-Leibniz 级数、Machin 公式、端序、机器精度与不稳定递推
实验环境	C 数值程序、quadmath 与统一运行日志

I. Gregory-Leibniz 级数计算圆周率

1 Problem 1: Gregory-Leibniz 级数计算圆周率

相关脚本：

- 本地 scripts/problem1.c
- GitHub scripts/problem1.c

1.1 待求问题

利用 Gregory-Leibniz 级数

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$$

计算 π 。

1.1.1 (1) 有限项计算

取大约 500000 项进行计算。

1.1.2 (2) 两种求和顺序比较

分别采用按正序求和与按逆序求和两种实现。

1.1.3 (3) 精度判断与原因说明

比较两种实现的精度差异，并说明哪一种更准确以及原因。

1.2 解决方式

本题的计算过程可表示为如下伪代码：

Input : number of terms N

Output: forward sum, backward sum, and absolute errors

```
sum_forward <- 0
for k from 0 to N - 1 do
    term <- (-1)^k / (2k + 1)
    sum_forward <- sum_forward + term
end for
```

```
sum_backward <- 0
for k from N - 1 down to 0 do
    term <- (-1)^k / (2k + 1)
```

```
sum_backward <- sum_backward + term
end for
```

```
pi_forward <- 4 * sum_forward
pi_backward <- 4 * sum_backward
compare both values with reference pi
```

程序同时输出 `double` 与 `__float128` 的结果，以区分截断误差和舍入误差的影响。

1.3 问题答案

1.3.1 (1) 有限项计算结果

取 500000 项时，结果如下。

方法	double	double 误差	float128	float128 误差
正序	3.141590653589692	2.000e-06	3.14159065358979324046	2.000e-06
逆序	3.141590653589793	2.000e-06	3.14159065358979324046	2.000e-06

1.3.2 (2) 两种求和顺序比较

逆序求和略优于正序求和，但两者的差距非常有限。对 `double` 而言，逆序结果在最后几位上更接近参考值；对 `__float128` 而言，两种顺序在当前展示精度下已几乎不可区分。

1.3.3 (3) 精度判断与原因说明

两种实现都无法达到 10 位小数精度，原因在于 Gregory-Leibniz 级数本身收敛过慢，主导误差来自级数截断而不是舍入顺序。日志还给出了达到 10 位小数大约需要 40000000000 项的估计，说明题目中给出的 500000 项主要用于观察误差机制，而不是为了真正完成 10 位十进制精度。

1.4 分析

本题的主导误差并不是求和顺序造成的舍入误差，而是级数本身的截断误差。对 Gregory-Leibniz 级数有

$$\left| \pi - 4 \sum_{k=0}^{N-1} \frac{(-1)^k}{2k+1} \right| \leq \frac{4}{2N+1},$$

当 $N=500000$ 时，这一上界本身就处于 10^{-6} 数量级。逆序求和确实略微减轻了舍入误差，但不能改变 Gregory-Leibniz 级数收敛过慢这一根本事实。

II. Machin 公式计算圆周率

2 Problem 2: Machin 公式计算圆周率

相关脚本：

- 本地 scripts/problem2.c
- GitHub scripts/problem2.c

2.1 待求问题

利用 Machin 公式

$$\frac{\pi}{4} = 4 \arctan\left(\frac{1}{5}\right) - \arctan\left(\frac{1}{239}\right)$$

以及反正切级数

$$\arctan(x) = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \dots$$

2.1.1 (1) 公式构造

利用 Machin 公式与反正切级数构造高精度圆周率计算方法。

2.1.2 (2) 精度目标

将 π 计算到 30 位小数。

2.2 解决方式

算法流程如下：

Input : $x_1 = 1/5$, $x_2 = 1/239$, tolerance τ

Output: high-precision approximation of π

```
function atan_series(x, tau):  
    sum <- 0  
    n <- 0  
    repeat  
        term <- (-1)^n * x^(2n+1) / (2n+1)  
        sum <- sum + term  
        n <- n + 1  
    until abs(term) < tau  
    return sum
```

```
a <- atan_series(1/5, 1e-35)
b <- atan_series(1/239, 1e-35)
pi <- 16a - 4b
```

实现使用 `__float128` 与 `quadmath.h`, 以保证级数截断精度满足 30 位小数要求。

2.3 问题答案

2.3.1 (1) 公式构造结果

程序得到：

$$\pi \approx 3.141592653589793238462643383279502797.$$

其中：

量	数值
<code>arctan(1/5)</code> 使用项数	25
<code>arctan(1/239)</code> 使用项数	8
<code>__float128</code> 参考值	3.141592653589793238462643383279502797
绝对误差	0.000000000000e+00

`arctan(1/5)` 需要 25 项, `arctan(1/239)` 需要 8 项, 说明 Machin 公式确实把问题转化成了两个快速收敛的小参数级数。

2.3.2 (2) 30 位小数结果

最终得到

$$\pi \approx 3.141592653589793238462643383279502797,$$

因此 Machin 公式在本题精度目标下完全满足 30 位小数要求。

2.4 分析

与第一题相比, Machin 公式的优势来自“小参数级数”的快速收敛。由于 $1/5$ 和 $1/239$ 都远小于 1, 高阶项会迅速衰减, 因此所需项数极少即可达到高精度目标。换言之, 本题展示的不是“换一种写法而已”, 而是通过恒等变换显著改善了数值算法的收敛效率。

III. 机器端序检测

3 Problem 3: 机器端序检测

相关脚本：

- 本地 scripts/problem3.c
- GitHub scripts/problem3.c

3.1 待求问题

编写 C 程序，判断当前计算机采用 big-endian 还是 little-endian 存储。

3.2 解决方式

判断过程可概括为：

Input : probe integer 0x01020304

Output: machine byte order

```
store integer in memory
view its address as byte array
if first byte == 0x04 then
    machine is little-endian
else if first byte == 0x01 then
    machine is big-endian
end if
```

3.3 问题答案

本机探针字节顺序为 04 03 02 01，因此当前机器采用 little-endian 存储。

3.4 分析

端序只描述多字节数据在内存中的排布方式，并不等同于处理器的全部体系结构特征。通过字节探针判断端序，是因为该方法直接考察同一整数在内存中的最低地址字节内容，因此具有明确而稳定的解释。

IV. 机器精度测量

4 Problem 4: 机器精度测量

相关脚本：

- 本地 scripts/problem4.c
- GitHub scripts/problem4.c

4.1 待求问题

编写程序，分别测量当前系统 float、double 与 quadruple precision 的 machine epsilon。

4.2 解决方式

本题采用经典的逐次二分方法：

Input : floating-point type T

Output: machine epsilon of T

```
eps <- 1
while 1 + eps / 2 != 1 do
  eps <- eps / 2
end while
return eps
```

上述过程分别对 float、double 和 __float128 执行。

4.3 问题答案

测得结果如下。

类型	machine epsilon
float	1.192092896e-07
double	2.220446049e-16
__float128	1.925929944e-34

三者都与理论参考值完全一致。

4.4 分析

对二进制浮点数，machine epsilon 满足

$$\varepsilon = 2^{1-p},$$

其中 p 是有效位数。float、double 与 __float128 的有效位数分别为 24、53、113，因此理论值恰好对应 2^{-23} 、 2^{-52} 与 2^{-112} 。本题结果说明逐次二分法能够正确探测浮点格式的最小分辨能力。

V. 不稳定递推与稳定改写

5 Problem 5: 不稳定递推与稳定改写

相关脚本：

- 本地 scripts/problem5.c
- GitHub scripts/problem5.c

5.1 待求问题

考虑序列

$$z_2 = 2, \quad z_{n+1} = 2^{n-\frac{1}{2}} \sqrt{1 - \sqrt{1 - 4^{1-n} z_n^2}}, \quad n = 2, 3, \dots$$

5.1.1 (1) 数值极限比较

比较数值递推结果与极限 π 。

5.1.2 (2) 不稳定原因解释

解释为何当 $n \geq 30$ 时会出现 rounding error propagation。

5.2 解决方式

程序同时实现题面中的直接递推与经有理化后的稳定递推：

Input : starting value $z_2 = 2$ and index range n

Output: direct recurrence, stabilized recurrence, and errors

```
for n from 2 upward do
  a <- 4^(1-n) * z_n^2
  z_direct[n+1] <- 2^(n-1/2) * sqrt(1 - sqrt(1 - a))
  z_stable[n+1] <- z_n * sqrt(2 / (1 + sqrt(1 - a)))
  compare both values with pi
end for
```

稳定形式利用了恒等变换

$$1 - \sqrt{1 - a} = \frac{a}{1 + \sqrt{1 - a}},$$

从而避免直接计算两个接近量之差。

5.3 问题答案

5.3.1 (1) 数值极限比较

代表性结果如下。

(n)	直接值	直接误差	稳定值	稳定误差
20	3.141594125195	1.472e-06	3.141592653571	-1.880e-11
25	3.142451272494	8.586e-04	3.141592653590	-1.865e-14
30	4.000000000000	8.584e-01	3.141592653590	-4.441e-16
31	8.000000000000	4.858e+00	3.141592653590	-4.441e-16
40	4096.000000000000	4.093e+03	3.141592653590	-4.441e-16

因此，直接递推在 $n=30$ 左右开始彻底失稳，而稳定改写后仍保持对 π 的收敛。

5.3.2 (2) 不稳定原因解释

当 n 增大时，直接递推中的差值 $1 - \sqrt{1 - a_n}$ 会变成两个极接近量之差，随后又被外层的放大因子迅速放大，因此会在 $n \geq 30$ 左右出现明显的 rounding error propagation。稳定改写通过有理化避免了这一误差通道。

5.4 分析

失稳的根源在于表达式

$$1 - \sqrt{1 - a_n}.$$

当 a_n 很小时， $\sqrt{1 - a_n}$ 与 1 极其接近，二者相减会造成严重相消误差；随后外层的放大因子 $2^{n-\frac{1}{2}}$ 又会把这部分误差迅速放大，从而使递推结果偏离 π 。稳定改写并没有改变问题的数学极限，而是改变了误差传播路径，因此能够显著抑制 rounding error propagation。